



Angular

Bootcamp Tercera Edición

Formularios en Angular

Módulo 6



Desglose del módulo

1. Reactive Forms
2. Signals



1. Reactive Forms

¿Qué es?

- Los Reactive Forms son la forma moderna de manejar formularios en Angular.
- A diferencia de los Template-driven forms donde el formulario vive en el HTML, en Reactive Forms **el formulario vive en el componente TypeScript.**



¿Cómo funciona?

- **FormControl**

Representa un solo campo del formulario. Tiene un valor, un estado (válido/inválido) y puede tener validaciones.

```
const nombre = new FormControl('',  
Validators.required);
```



¿Cómo funciona?

- **FormGroup**

Agrupar varios FormControl bajo un mismo objeto. Representa el formulario completo.

```
const form = new FormGroup({  name: new  
  FormControl('', Validators.required),  
  email: new FormControl('',  
    [Validators.required,  
    Validators.email]))});
```



¿Cómo conectarlo al template HTML?

- [formGroup] — conecta el form del componente con el <form> del template.
- FormControlName — conecta cada input con su FormControl por nombre.
- (ngSubmit) — dispara la función cuando el usuario envía el formulario.

```
<form [formGroup]="form" (ngSubmit)="guardar()">
  <mat-form-field>
    <input matInput FormControlName="name"
placeholder="Nombre">
  </mat-form-field>
  <button mat-raised-button type="submit">
    Guardar
  </button>
</form>
```



Validaciones

Las validaciones se definen en el componente, los mensajes se muestran en el template.

En el componente

```
form = this.fb.group({  
  name: ['', [Validators.required,  
Validators.minLength(3)]],  
  email: ['', [Validators.required,  
Validators.email]]  
});
```




Validaciones

Las validaciones se definen en el componente, los mensajes se muestran en el template.

En el componente

```
form = this.fb.group({  
  name: ['', [Validators.required,  
Validators.minLength(3)]],  
  email: ['', [Validators.required,  
Validators.email]]  
});
```

Validaciones

```
<mat-form-field>
  <input matInput formControlName="name" placeholder="Nombre">
  @if (form.get('name')?.hasError('required') && form.get('name')?.touched) {
    <mat-error>El nombre es obligatorio</mat-error>
  }
  @if (form.get('name')?.hasError('minlength') && form.get('name')?.touched) {
    <mat-error>Mínimo 3 caracteres</mat-error>
  }
</mat-form-field>
```

Full Name

Fullname is required

Username

Username is required





patchValue vs setValue

- setValue — requiere que se envíen todos los campos del formulario. Si falta uno, rompe.
- patchValue — solo actualiza los campos que se le envíen.

```
ngOnInit() {  
  const id = this.route.snapshot.params['id'];  
  
  if (id) {  
    this.isEdit.set(true);  
    this.studentService.getById(id).subscribe(data => {  
      this.student.set(data);  
      this.form.patchValue(data);  
    });  
  }  
}
```



```
<form [formGroup]="form" (ngSubmit)="guardar()">
  <mat-form-field>
    <input matInput formControlName="name" placeholder="Nombre">
  </mat-form-field>
  <mat-form-field>
    <input matInput formControlName="email" placeholder="Email">
  </mat-form-field>
  <button mat-raised-button color="primary" type="submit">
    {{ isEdit() ? 'Actualizar' : 'Crear' }}
  </button>
</form>
```



```
guardar() {  
  if (this.form.invalid) return;  
  this.loading.set(true);  
  
  const accion = this.isEdit()  
    ? this.studentService.update(this.student().!.id, this.form.value)  
    : this.studentService.create(this.form.value);  
  
  accion.subscribe({  
    next: () => this.router.navigate(['/students']),  
    error: () => this.loading.set(false)  
  });  
}
```



2. Signals

¿Qué es?

- Angular necesita saber cuándo cambió un dato para actualizar el template. Los **signals** permiten que en vez que Angular busque los cambios, el dato mismo avisa cuando cambió.
- Angular solo actualiza las partes del template que dependen de ese Signal específico.



```
loading = signal(false);
errorMsg = signal<string | null>(null);
students = signal<IStudent[]>([]);

cargar() {
  this.loading.set(true);      // antes de la petición
  this.errorMsg.set(null);

  this.studentService.getAll().subscribe({
    next: (data) => {
      this.students.set(data);
      this.loading.set(false); // cuando llega la data
    },
    error: () => {
      this.errorMsg.set('No se pudieron cargar los estudiantes.');
```



```
@if (loading()) {  
  <mat-spinner />  
}  
  
@if (errorMsg()) {  
  <mat-error>{{ errorMsg() }}</mat-error>  
}  
  
@if (!loading() && !errorMsg()) {  
  @for (student of students(); track student.id) {  
    ...  
  }  
}
```